

TINY MIPS CPU DESIGN

Joyce Berdkan

Electrical and Computer Engineering
George Washington University - GWU
District of Columbia, USA
j.berdkan@gwu.edu

Syed Hasan

Electrical and Computer Engineering
George Washington University - GWU
District of Columbia, USA
Syed.hasan@gwu.edu

Abstract - This paper presents the design, implementation, and verification of a multi-cycle Tiny MIPS CPU using Verilog HDL. The proposed processor follows a resource-efficient architecture in which a single datapath is reused across multiple cycles to execute instructions. The design integrates a finite-state control unit with a modular datapath comprising an ALU, register file, memory interface, and control logic. Functional verification is performed using self-checking testbenches and waveform analysis. The design is further evaluated through synthesis, scan insertion, and physical design stages, providing insights into timing, area, and power characteristics. The results demonstrate a correct and scalable CPU implementation suitable for educational and foundational processor design exploration.

Index Terms – MIPS, Verilog HDL, CPU Design, Datapath, Control Unit, VLSI.

I. INTRODUCTION

Processor design is a fundamental aspect of digital system engineering, bridging theoretical computer architecture with practical hardware implementation. The MIPS architecture, known for its simplicity and modularity, serves as an ideal platform for understanding core processor design concepts.

This project implements a **multi-cycle Tiny MIPS CPU**, where instruction execution is divided into multiple stages such as fetch, decode, execute, memory access, and write-back. Unlike single-cycle architectures, the multi-cycle approach reuses hardware resources, reducing area and power consumption at the cost of increased execution cycles.

The system integrates a unified memory interface, instruction register (IR), memory data register (MDR), ALU, and control unit governed by a finite-state machine (FSM). The design flow includes RTL modeling, functional verification, synthesis, design-for-test (DFT), and physical layout generation, providing a complete ASIC design experience.

II. SYSTEM OVERVIEW (SYSTOLIC EXECUTION MODEL)

A. Datapath Overview

The datapath consists of interconnected hardware modules responsible for executing instructions. As illustrated in the datapath diagram, key components include:

- Program Counter (PC) and PCNext logic
- Instruction Register (IR)
- Register File (8 registers: \$s0–\$s7)
- Arithmetic Logic Unit (ALU)
- ALUOut register for intermediate storage
- Memory interface (shared instruction/data memory)
- Multiplexers for operand selection

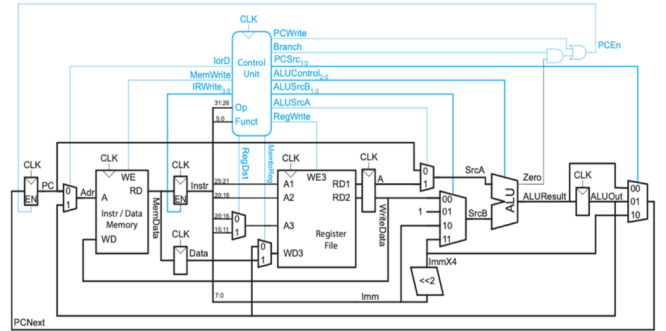


Figure 1: Data Path Diagram

B. Control Unit

The control unit generates signals such as:

- PCWrite, MemWrite, RegWrite
- ALUSrcA, ALUSrcB
- MemtoReg, RegDst
- PCSrc

These signals coordinate datapath operations across multiple clock cycles.

C. Instruction Set

The CPU supports a subset of MIPS instructions including:

- **R-type:** add, sub, and, or, slt
- **I-type:** addi, lb, sb
- **Control flow:** beq, j

The **instruction encoding table** defines opcode and function fields.

Instruction	Function	Encoding	op	funct
add \$1, \$2, \$3	addition: $S1 \rightarrow S2 + S3$	R	000000	100000
sub \$1, \$2, \$3	subtraction: $S1 \rightarrow S2 - S3$	R	000000	100010
and \$1, \$2, \$3	bitwise and: $S1 \rightarrow S2 \text{ and } S3$	R	000000	100100
or \$1, \$2, \$3	bitwise or: $S1 \rightarrow S2 \text{ or } S3$	R	000000	100101
slt \$1, \$2, \$3	set less than: $S1 \rightarrow 1 \text{ if } S2 < S3$ $S1 \rightarrow 0 \text{ otherwise}$	R	000000	101010
addi \$1, \$2, imm	add immediate: $S1 \rightarrow S2 + \text{imm}$	I	001000	n/a
beq \$1, \$2, imm	branch if equal: $PC \rightarrow PC + \text{imm}^a$	I	000100	n/a
j destination	jump: $PC_destination^a$	J	000010	n/a
lb \$1, imm(\$2)	load byte: $S1 \rightarrow \text{mem}[S2 + \text{imm}]$	I	100000	n/a
sb \$1, imm(\$2)	store byte: $\text{mem}[S2 + \text{imm}] \rightarrow S1$	I	110000	n/a

Figure 2: Instruction Encoding Table

III. CONTROL DESIGN (FSM)

The processor operation is governed by a **finite-state machine**, which sequences instruction execution.

The **FSM diagram** shows states such as:

1. Instruction Fetch
2. Instruction Decode
3. Execution
4. Memory Access
5. Write-back

Each state activates specific control signals to drive datapath components. For example:

- Fetch $\rightarrow$ PC update + memory read
- Decode $\rightarrow$ register access
- Execute $\rightarrow$ ALU operation
- Memory $\rightarrow$ load/store
- Write-back $\rightarrow$ register update

This structured sequencing ensures correct multi-cycle execution.

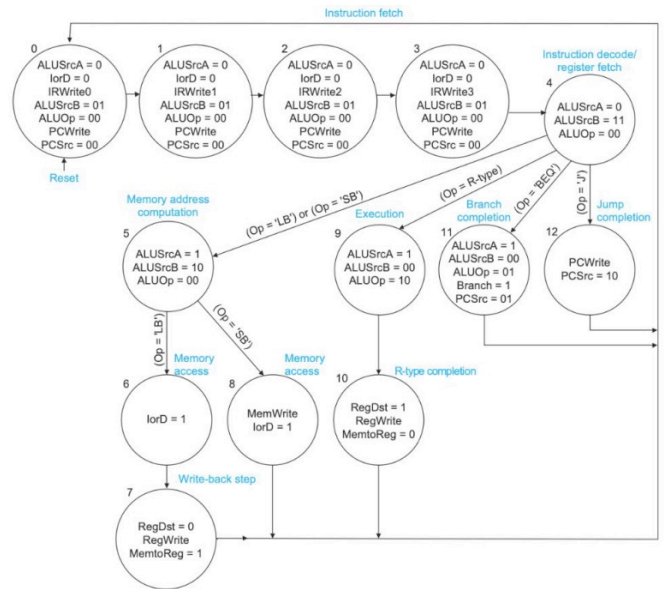


Figure 3: FSM Diagram

IV. RTL IMPLEMENTATION

The Tiny MIPS CPU was implemented in Verilog HDL using a modular RTL design approach. The complete processor is divided into a top-level integration module, a finite-state controller, a datapath, ALU control logic, and supporting sequential and combinational blocks.

A. Top-Level CPU Module: mips.v

The mips.v module is the top-level processor wrapper. It connects the controller, ALU control unit, and datapath. The external interface includes the clock, reset, memory data input, memory read/write control signals, memory address, and write-data output.

The controller receives the opcode field from the instruction and the zero flag from the datapath. It then generates control signals such as memread, memwrite, alusrcA, memtoReg, iord, pcen, regwrite, regdst, pcsrc, alusrcB, aluop, and irwrite. The ALU control module converts aluop and the instruction function field into the final ALU operation code.

B. Controller Module: controller.v

The controller is designed as a finite-state machine that sequences the multi-cycle execution of each instruction. It contains states for instruction fetch, decode, memory address calculation, memory read, memory write, R-type execution, R-type write-back, branch completion, jump completion, addi execution, and addi write-back.

The instruction fetch process is divided into four states because the memory interface is 8 bits wide while the instruction word is 32 bits. Each fetch state loads one byte

of the instruction into the instruction register using the 4-bit irwrite signal.

The controller supports the following opcodes:

Instruction Type	Opcode
R-type	000000
addi	001000
beq	000100
jump	000010
lb	100000
sb	101000

Figure 4: Table represents instruction Type and Opcode.

C. Datapath Module: datapath.v

The datapath module implements the main execution hardware of the CPU. It contains the program counter, instruction register, memory data register, register file, ALU, ALUOut register, multiplexers, and zero detection logic.

D. ALU Module: alu.v

The ALU performs the arithmetic and logical operations required by the supported Tiny MIPS instruction set. It accepts two 8-bit operands and a 3-bit ALU control input.

ALU Control	Operation
000	AND
001	OR
010	ADD
110	SUB
111	SLT

Figure 5: Table represents ALU Control and Operation.

The SLT operation uses signed comparison and produces 8'd1 when operand a is less than operand b; otherwise, it produces 8'd0.

E. ALU Control Module: alucontrol.v

The ALU control block translates the high-level aluop signal from the controller and the instruction funct field into the 3-bit ALU operation code.

For load, store, addi, fetch, and address calculation, aluop = 00 selects addition. For branch-equal, aluop = 01 selects subtraction. For R-type instructions, aluop = 10 enables decoding of the function field.

F. Register File Module: regfile.v

The register file contains eight 8-bit registers corresponding to the simplified register set \$s0 through \$s7. It supports two asynchronous read ports and one synchronous write port.

G. Sequential Storage Modules

Three sequential modules are used throughout the datapath:

Module	Function
flop.v	8-bit positive-edge D flip-flop
flopen.v	8-bit flip-flop with enable
flopenr.v	8-bit flip-flop with enable and reset

Figure 6: Table Represents Sequential Storage Modules and its functions

The program counter uses flopenr so that it can be reset to address 0x00 and updated only when pcen is asserted.

H. Multiplexer Modules

The design uses three multiplexer modules:

Module	Width	Purpose
mux2.v	8-bit	Selects between two datapath inputs
mux4.v	8-bit	Selects among four datapath inputs
mux23.v	3-bit	Selects register destination address

Figure 7: Table Represents Multiplexer Modules, width and its purpose

These multiplexers allow the same ALU and datapath hardware to be reused across different instruction types.

I. Zero Detection Module: zerodetect.v

The zero detection module checks whether the ALU result is equal to zero. Its output is used by the controller during beq execution. If the subtraction result is zero, the branch condition is true and the PC is updated.

J. Memory Module: ram.v

The ram.v module models a 256-word, 8-bit memory used for instruction and data storage during simulation. It is initialized using ram.dat. The memory performs reads and writes on the negative clock edge, which separates memory access timing from positive-edge datapath register updates.

I. RTL Hierarchy Summary

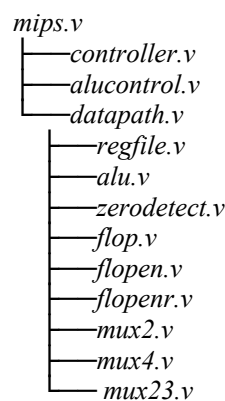


Figure 7: Figure Represents RTL Hierarchy

This modular RTL structure improves readability, simplifies debugging, and allows each block to be individually verified before complete CPU-level simulation.

V. FUNCTIONAL VERIFICATION

A. Testbench Overview

A self-checking Verilog testbench (tb_mips_final.v) was developed to verify the functionality of the Tiny MIPS CPU. The testbench applies a sequence of instructions stored in memory (ram.dat) and automatically validates the correctness of register values, memory operations, and instruction execution.

The verification strategy includes:

- Arithmetic and logical instruction validation
- Load/store memory access verification
- Branch and jump control flow verification
- Instruction coverage tracking using internal flags

The testbench reports pass/fail status using automated checks without manual inspection.

B. Waveform Analysis

The simulation waveform provides detailed visibility into the multi-cycle execution of instructions. Internal control and datapath signals were probed to validate correct sequencing of operations.

1. Multi-Cycle Execution

The waveform shows that each instruction is executed across multiple clock cycles, corresponding to:

- Instruction Fetch (multiple cycles due to byte-wise memory access)
- Instruction Decode / Register Fetch
- Execution (ALU operation or address calculation)
- Memory Access (load/store operations)
- Write-back (register update)

The FSM state transitions confirm proper sequencing of these stages.

2. Datapath Behavior

The following datapath signals verify correct operation:

- pc and nextpc show proper program flow
- instr confirms correct instruction fetch
- aluresult and aluout validate arithmetic and logical operations
- zero signal correctly drives branch decisions

3. Control Signal Verification

The control unit signals demonstrate correct coordination of datapath operations:

- regwrite enables register updates during write-back
- memread and memwrite control memory access

- pcen and pcsorce correctly update the program counter
- alusrc and alusrcb select ALU inputs appropriately
- irwrite controls instruction register loading

These signals confirm correct FSM-driven control behavior.

4. Branch and Jump Verification

Branch (BEQ) and jump (JUMP) instructions are validated through:

- Correct assertion of pcen and pcsorce
- Proper evaluation of the zero flag
- Accurate updates of the program counter

The resulting register values (\$s5, \$s6) confirm correct control flow execution.

5. Memory Access Verification

Load (LB) and store (SB) operations are validated through:

- memread and memwrite signals
- Correct address generation (adr)
- Proper data transfer (memdata, writedata)

The final memory contents confirm successful data movement.

Summary:

The functional verification demonstrates that the Tiny MIPS CPU correctly implements all required instruction types and control behaviors. The self-checking testbench, combined with waveform analysis, confirms:

- Correct multi-cycle execution
- Proper datapath and control interaction
- Accurate arithmetic, logical, memory, and control operations

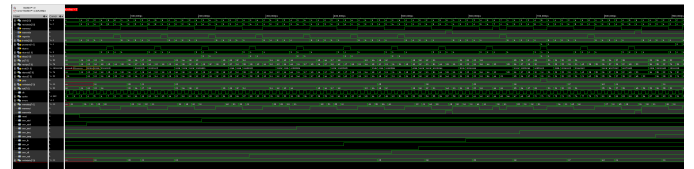


Figure 8: Waveforms of Tiny MIPS CPU

C. Simulation Log Results

The simulation results confirm correct processor functionality. The final output log shows:

- Correct register values after execution:
 - \$s1 = 5, \$s2 = 3, \$s3 = 8, \$s4 = 8
 - \$s5 = 1 (branch/jump execution result)
 - \$s6 = 99 (jump execution result)
 - \$s7 = 1 (SLT result)
- Correct memory operations:
 - mem[100] = 8 (store operation)
 - mem[101] = 1 (final result storage)
- Complete instruction coverage:
 - ADDI, ADD, SUB, AND, OR, SLT

- LB, SB
- BEQ, JUMP

The testbench reports: TB_MIPS_FINAL PASSED
All arithmetic, logic, load/store, branch, jump, and coverage checks passed.

This confirms that all functional requirements of the Tiny MIPS CPU are satisfied.

```

=====
Created default SHM database waves
xcelium> probe -create -sha tb_mips_final.dut.cont.state tb_mips_final.dut.cont.\
nextstate tb_mips_final.dut.cont.memread tb_mips_final.dut.cont.memwrite tb_mips\
_final.dut.cont.regwrite tb_mips_final.dut.cont.firwrite tb_mips_final.dut.cont.p\
source tb_mips_final.dut.cont.alusrc tb_mips_final.dut.cont.alusrcb tb_mips_fi\
nal.dut.cont.aluop tb_mips_final.dut.dp.pc tb_mips_final.dut.dp.nextpc tb_mips_f\
inal.dut.dp.instr tb_mips_final.dut.dp.aluresult tb_mips_final.dut.dp.aluout tb_\
mips_final.dut.dp.zero tb_mips_final.dut.dp.writedata tb_mips_final.adr tb_mips_\
final.clk tb_mips_final.cycles tb_mips_final.errors tb_mips_final.memdata tb_mip\
s_final.memread tb_mips_final.memwrite tb_mips_final.reset tb_mips_final.saw_add\
tb_mips_final.saw_addi tb_mips_final.saw_and tb_mips_final.saw_beq tb_mips_fina\
l.saw_jump tb_mips_final.saw_lb tb_mips_final.saw_or tb_mips_final.saw_sb tb_mip\
s_final.saw_slt tb_mips_final.saw_sub tb_mips_final.writedata
Created probe 1
xcelium> run
xsim: *H,SHMDFU: $sha_open - file waves.sha already in use (SHM database).
File: ./ece6250/lab8/simulation/testbench/tb_mips_final.v, line = 76, pos = 12
Scope: tb_mips_final
Time: 0 FS + 0

xsim: *H,SHMNOSD: $sha_probe - SHM file not opened with $sha_open.
File: ./ece6250/lab8/simulation/testbench/tb_mips_final.v, line = 77, pos = 13
Scope: tb_mips_final
Time: 0 FS + 0

===== FINAL REGISTER CHECKS =====
PRGS: #s1 addi result = 5
PRGS: #s2 addi result = 3
PRGS: #s3 add result = 8
PRGS: #s4 lb result = 8
PRGS: #s5 branch/jump program value = 1
PRGS: #s6 jump program value = 99
PRGS: #s7 slt result = 1

===== FINAL MEMORY CHECKS =====
PRGS: mem[100] store result = 8
PRGS: mem[101] final store result = 1

===== INSTRUCTION COVERAGE CHECKS =====
PRGS: coverage seen for RADDI
PRGS: coverage seen for ADD
PRGS: coverage seen for SUB
PRGS: coverage seen for RND
PRGS: coverage seen for OR
PRGS: coverage seen for SLT
PRGS: coverage seen for LB
PRGS: coverage seen for SB
PRGS: coverage seen for BEQ
PRGS: coverage seen for JUMP

=====
TB_MIPS_FINAL PASSED
All arithmetic, logic, load/store, branch, jump, and coverage checks passed.
=====

Simulation complete via #finish(1) at time 2225 NS + 0
./ece6250/lab8/simulation/testbench/tb_mips_final.v:144      #finish:\r

```

Figure 9: Simulation Log of Tiny MIPS CPU

VI. SYNTHESIS